



ASSIGNMENT NO.3

OBJECT ORIENTED PROGRAMMING

Submitted by:

Vigyat Manktala

1024150373

2034

Q1. Write a program to implement (a) pointer to an object (b) this pointer.
Practice both '.' (dot operator) and '→' (arrow operator).

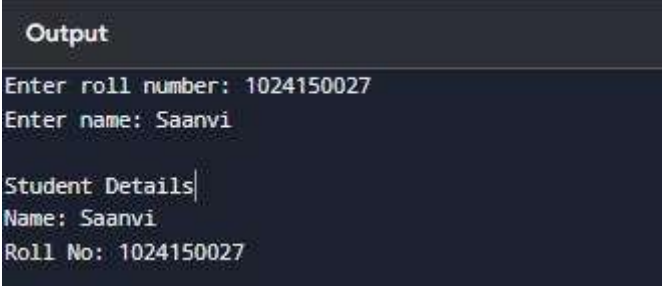
```
#include <iostream>
#include <string>
using namespace std;

class student
{
    int roll_no;
    string name;

public:
    void getdata()
    {
        cout << "Enter roll number: ";
        cin >> roll_no;

        cout << "Enter name: ";
        cin >> name;
    }

    void display()
    {
        cout << "\nStudent Details\n";
        cout << "Name: " << name << endl;
        cout << "Roll No: " << roll_no << endl;
    }
};
```

A screenshot of a terminal window with a dark background. The title bar at the top says "Output". The text in the terminal shows the program's execution: it prompts for a roll number and name, receives the inputs "1024150027" and "Saanvi", and then displays the student details. The output text is as follows:

```
Enter roll number: 1024150027
Enter name: Saanvi

Student Details
Name: Saanvi
Roll No: 1024150027
```

```
int main()
{
    student s;
    student* ptr = &s;

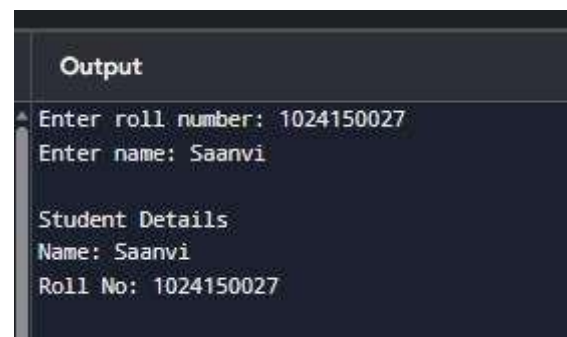
    ptr->getdata();
    ptr->display();

    return 0;
}
```

```
#include <iostream>
#include <string>
using namespace std;
```

```
class student
{
    int roll_no;
    string name;

public:
    void getdata()
    {
        cout << "Enter roll number: ";
        cin >> roll_no;
        cout << "Enter name: ";
        cin >> name;
    }
}
```



```
Output
Enter roll number: 1024150027
Enter name: Saanvi

Student Details
Name: Saanvi
Roll No: 1024150027
```

```

void display()
{
    cout << "\nStudent Details\n";
    cout << "Name: " << name << endl;
    cout << "Roll No: " << roll_no << endl;
}
};

```

```

int main()
{
    student s;
    s.getdata();
    s.display();

    return 0;
}

```

2. Write a program to swap private values of two classes using a friend function.

```

#include <iostream>
using namespace std;

```

```

class B; // forward declaration
class A
{
    int x;

```

public:

void set(int a)

{

 x = a;

}

void show()

{

 cout << "Value of A: " << x << endl;

}

friend void swap(A &, B &);

};

class B

{

 int y;

public:

void set(int b)

{

 y = b;

}

void show()

{

 cout << "Value of B: " << y << endl;

}

Output

Before swap:

Value of A: 5

Value of B: 10

After swap:

Value of A: 10

Value of B: 5

```
    friend void swap(A &, B &);  
};
```

```
void swap(A &a, B &b)  
{  
    int temp;  
    temp = a.x;  
    a.x = b.y;  
    b.y = temp;  
}
```

```
int main()  
{  
    A a;  
    B b;  
  
    a.set(5);  
    b.set(10);  
  
    cout << "Before swap:" << endl;  
    a.show();  
    b.show();  
  
    swap(a, b);  
    cout << "After swap:" << endl;  
    a.show();  
    b.show();  
}
```

```
    return 0;  
}
```

3. Write a program to add data objects of two different classes using friend Functions.

```
#include <iostream>  
using namespace std;
```

```
class B;
```

```
class A
```

```
{  
    int x;
```

```
public:
```

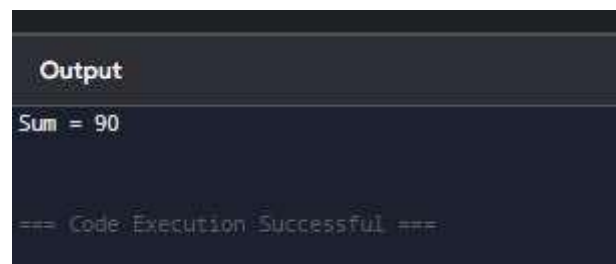
```
    void set(int a)
```

```
{  
    x = a;  
}
```

```
    friend void add(A, B);  
};
```

```
class B
```

```
{  
    int y;
```



```
Output  
Sum = 90  
  
=== Code Execution Successful ===
```

```

public:
    void set(int b)
    {
        y = b;
    }

    friend void add(A, B);
};

void add(A a, B b)
{
    int sum;
    sum = a.x + b.y;
    cout << "Sum = " << sum << endl;
}

int main()
{
    A a;
    B b;

    a.set(50);
    b.set(40);
    add(a, b);

    return 0;
}

```

4. Write a program to demonstrate the working of friend class.


```
#include <iostream>
using namespace std;
```

```
class B;
class A
{
    int x;
```

```
public:
    A()
    {
        x = 45;
    }

    friend class B;
};
```

```
class B
{
public:
    void show(A obj)
    {
        cout << "Value of x = " << obj.x << endl;
    }
};
```



Output

▲ Value of x = 45

=== Code Execution Successful ===

```

int main()
{
    A a;
    B b;

    b.show(a);

    return 0;
}

```

5. Write a program using Array of Objects to display area of multiple rectangles.

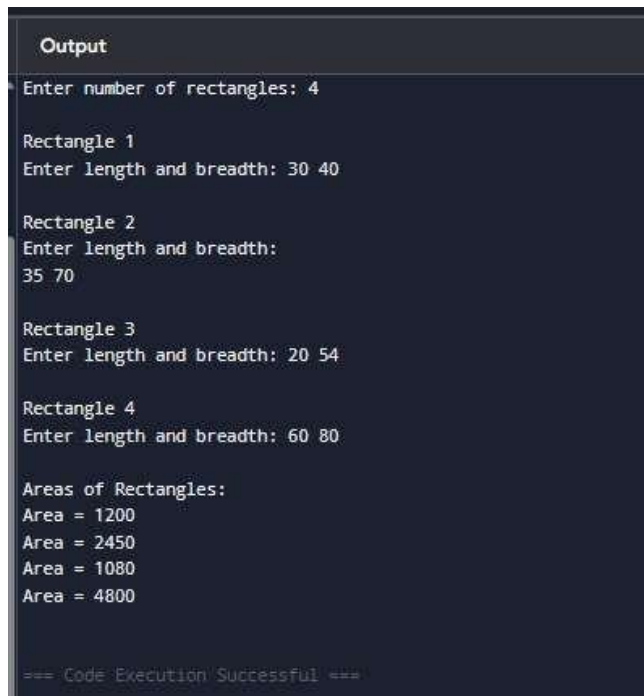
```

#include <iostream>
using namespace std;

class Rectangle
{
    int length, breadth;

public:
    void getData()
    {
        cout << "Enter length and breadth: ";
        cin >> length >> breadth;
    }
}

```



```

Output
Enter number of rectangles: 4

Rectangle 1
Enter length and breadth: 30 40

Rectangle 2
Enter length and breadth:
35 70

Rectangle 3
Enter length and breadth: 20 54

Rectangle 4
Enter length and breadth: 60 80

Areas of Rectangles:
Area = 1200
Area = 2450
Area = 1080
Area = 4800

=== Code Execution Successful ===

```

```
void displayArea()
{
    cout << "Area = " << length * breadth << endl;
}
};
```

```
int main()
{
    int n;
    cout << "Enter number of rectangles: ";
    cin >> n;
```

```
    Rectangle r[n];
```

```
    for (int i = 0; i < n; i++)
    {
        cout << "\nRectangle " << i + 1 << endl;
        r[i].getData();
    }
```

```
    cout << "\nAreas of Rectangles:\n";
```

```
    for (int i = 0; i < n; i++)
    {
        r[i].displayArea();
    }
```

```
    return 0;
}
```

6. Write a program to define function cube() as inline for calculating cube of a number.

```
#include <iostream>
using namespace std;
```

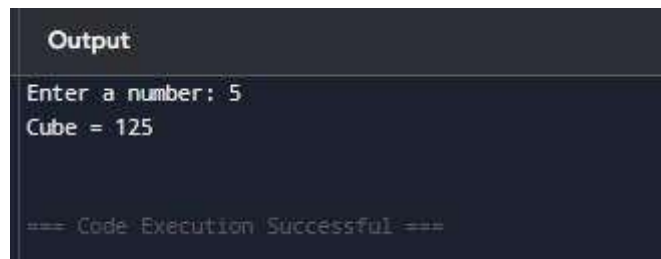
```
inline int cube(int x)
{
    return x * x * x;
}
```

```
int main()
{
    int num;

    cout << "Enter a number: ";
    cin >> num;

    cout << "Cube = " << cube(num) << endl;

    return 0;
}
```

A screenshot of a terminal window with a dark background. The title bar says "Output". The text inside shows the program's execution: "Enter a number: 5" followed by "Cube = 125". At the bottom, it says "=== Code Execution Successful ===".

```
Output
Enter a number: 5
Cube = 125

=== Code Execution Successful ===
```

7. Write a program to pass an object as an argument and return the object from a function.

a. Use pass-by-value

```
#include <iostream>
using namespace std;
```

```

class Number
{
public:
    int x;

    void get(int a)
    {
        x = a;
    }

    void show()
    {
        cout << "Value = " << x << endl;
    }
};

```

```

Number increment(Number n)

```

```

{
    n.x = n.x + 25;
    return n;
}

```

```

int main()

```

```

{
    Number n1, n2;

    n1.get(5);

    n2 = increment(n1);
}

```

Output

```

Original object: Value = 5
Returned object: Value = 30

```

```

=== Code Execution Successful ===

```

```

    cout << "Original object: ";
    n1.show();

    cout << "Returned object: ";
    n2.show();

    return 0;
}

```

b. Use pass-by reference

```

#include <iostream>
using namespace std;

class Number
{
public:
    int x;
    void get(int a)
    {
        x = a;
    }

    void show()
    {
        cout << "Value = " << x << endl;
    }
};

```

```

Output
After function call: Value = 30

=== Code Execution Successful ===

```

```
Number& increment(Number &n)
```

```
{
```

```
    n.x = n.x + 25;
```

```
    return n;
```

```
}
```

```
int main()
```

```
{
```

```
    Number n1;
```

```
    n1.get(5);
```

```
    increment(n1);
```

```
    cout << "After function call: ";
```

```
    n1.show();
```

```
    return 0;
```

```
}
```